86 AW DAILY MC AIRCRAFT FORECASTING MODEL

BACKGROUND

There are many on-going efforts to predict aircraft readiness and create metrics that will enable us to plan for future requirements. One example of this effort is a collaboration beteen the 603rd Air Operations Center research analysts with Palantir/Envision to develop a data pipeline to extract and utlize data from the MISs and develop or incorporate to the readiness model. Key data that is currently being utilize is MC rates, scheduled maintenance intervals, break rates, fix rates, among others that have yet to be decided upon. while this approach may well be the avenue that will provide us with statistically sound product, it might become "too complex" for the typical maintenance managers and analysts to utilize.

This effort is a much simpler and straightforward method is designed to supplement or add to the on-going 603rd AOC project. We aim to utilize historical daily mission capable aircraft and forecast for future requirements by employing Time-Series analysis and the ARIMA (Auto-regressive Integrated Moving Average) model.

For this analysis, we will only be using historical data from the 86AW C-130J daily aircraft status dating back to 2020. We are doing this because the C-130 data from the G081 -- the authorized data entry system for mobility aircraft, is much simpler and requires data manipulation and transformation compared to the fighter aircraft MIS.

We hope to be able to develop a viable product that can be used not only to supplement the readiness model for the 603rd but also for field managers and analysts to forecast daily aircraft availability.

1. EXPLORATORY DATA ANALYSIS

In [1]:
```
# LOAD DEPENDENCIES
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('status_2020.csv', parse_dates=['date'])
df.sort_values(by='date', axis=0, ascending=True)

print(df.head())
print(df.tail())
```

```
        date  daily_poss  daily_mc  mc_acft   mc_rate
0 2020-01-01       336.0     264.0    11.00  0.785714
1 2020-01-02       336.0     248.9    10.37  0.740774
2 2020-01-03       336.0     308.0    12.83  0.916667
3 2020-01-04       336.0     317.3    13.22  0.944345
```

```
4 2020-01-05       336.0     293.7    12.24  0.874107
          date  daily_poss  daily_mc  mc_acft   mc_rate
879 2022-05-29       288.0     256.0    10.67  0.888889
880 2022-05-30       288.0     238.0     9.92  0.826389
881 2022-05-31       288.0     202.3     8.43  0.702431
882 2022-06-01       288.0     201.1     8.38  0.698264
883 2022-06-02       288.0     195.0     8.13  0.677083
```

Discussion

As we can see above, our dataset is comprised of 883 observations with five different columns; "date", "daily_poss", "daily_mc", and "mc_rate".

- date ranges from 1-Jan 2020 to 2-Jun 2022
- daily_poss is the possessed hours for each day and can be further explaines the "number of aircraft multiplied by 24 hours in a day"
- daily_mc is the Mission Capable hours by daya based off the possessed hours
- mc_acft is the result of dividing the mc_hours by 24 hours
- mc_rate is the result of the daily_mc divided by the daily_poss

The value that we are most interested in is the mc_acft and We will further describe the dataset above using descriptive statistics.

In [2]:

```
# DESCRIPTIVE STATISTICS

print(f'Min: {df.mc_acft.min()}')
print(f'Quartile 25%: {df.mc_acft.quantile(.25)}')
print(f'Quartile 50%: {df.mc_acft.quantile(.50)}')
print(f'Quartile 75%: {df.mc_acft.quantile(.75)}')
print(f'Max: {df.mc_acft.max()}')
print(f'Mean: {df.mc_acft.mean()}')
print(f'Median: {df.mc_acft.median()}')
print(f'Mode: {df.mc_acft.mode().values[0]}')
print(f'STD: {df.mc_acft.std()}')
print(f'Skewness: {df.mc_acft.skew()}')
print(f'Kurtosis: {df.mc_acft.kurtosis()}')
```

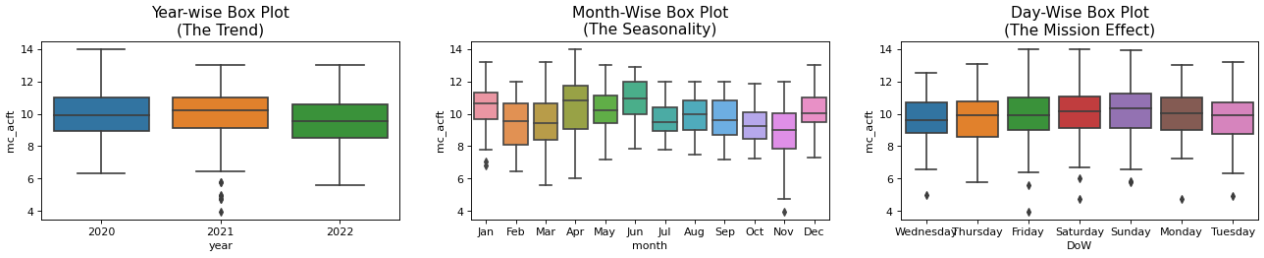
```
Min: 3.97
Quartile 25%: 8.9375
Quartile 50%: 10.0
Quartile 75%: 11.0
Max: 14.0
Mean: 9.909445701357464
Median: 10.0
Mode: 11.0
STD: 1.5327878317797836
Skewness: -0.30240785355114963
Kurtosis: 0.11896714456512969
```

Discussion

The figures above give us a pretty good description of the daily mc_acft. To begin, the minimum MC acft is less than 4 while the maximumn is 14.0. Both the mean (average) and median (most frequent) is roughly at 10 aircraft per day. The STD (standard deiation) is slightly greater than 1.5 meaning the

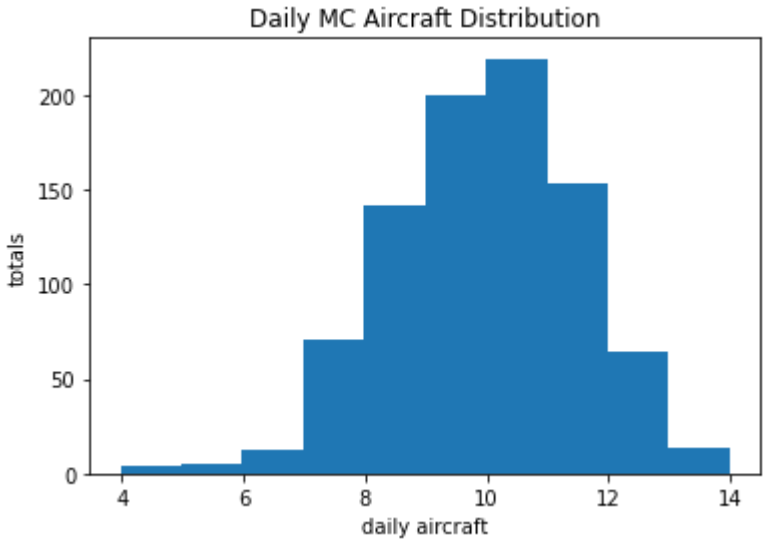
spread of the data is slightly greater than normally expected. The last two descriptions, skewness and kurtosis, tells us that our data distributiion is skewed slightly to the left of the normal curve and slighltly taller as depicted by the > 0 kurtosis. We can visualize the above values to understand the data further.

In [3]:
```
# Box and Whisker Plot of Daily MC Aircraft

# Prepare data
df['year'] = [d.year for d in df.date]
df['month'] = [d.strftime('%b') for d in df.date]
df['DoW'] = df['date'].dt.day_name()
years = df['year'].unique()

# Draw Plot
fig, axes = plt.subplots(1, 3, figsize=(20,3), dpi= 80)
sns.boxplot(x='year', y='mc_acft', data=df, ax=axes[0])
sns.boxplot(x='month', y='mc_acft', data=df, ax=axes[1])
sns.boxplot(x='DoW', y='mc_acft', data=df, ax=axes[2])

# Set Title
axes[0].set_title('Year-wise Box Plot\n(The Trend)', fontsize=14);
axes[1].set_title('Month-Wise Box Plot\n(The Seasonality)', fontsize=14)
axes[2].set_title('Day-Wise Box Plot\n(The Mission Effect)', fontsize=14)
plt.show()
```



The figure above shows the box plot of the mc aircraft distribution in three different axes -- year, month, and day of the week. The line in the middle indicates the mean. Above the mean is the upper quartile and below the mean is the lower quartile. Noteworthy of this plot is the number of outliers past the lower range of the distribution. It is important to keep note of this as we might have to perform some form of outlier treatment further in our analysis.

In [4]:
```
# DESCRIPTIVE PLOTS

# Histogram of Daily MC Aircraft
df.hist('mc_acft', grid= False)
plt.title('Daily MC Aircraft Distribution')
plt.xlabel('daily aircraft')
plt.ylabel('totals')
plt.show()
```



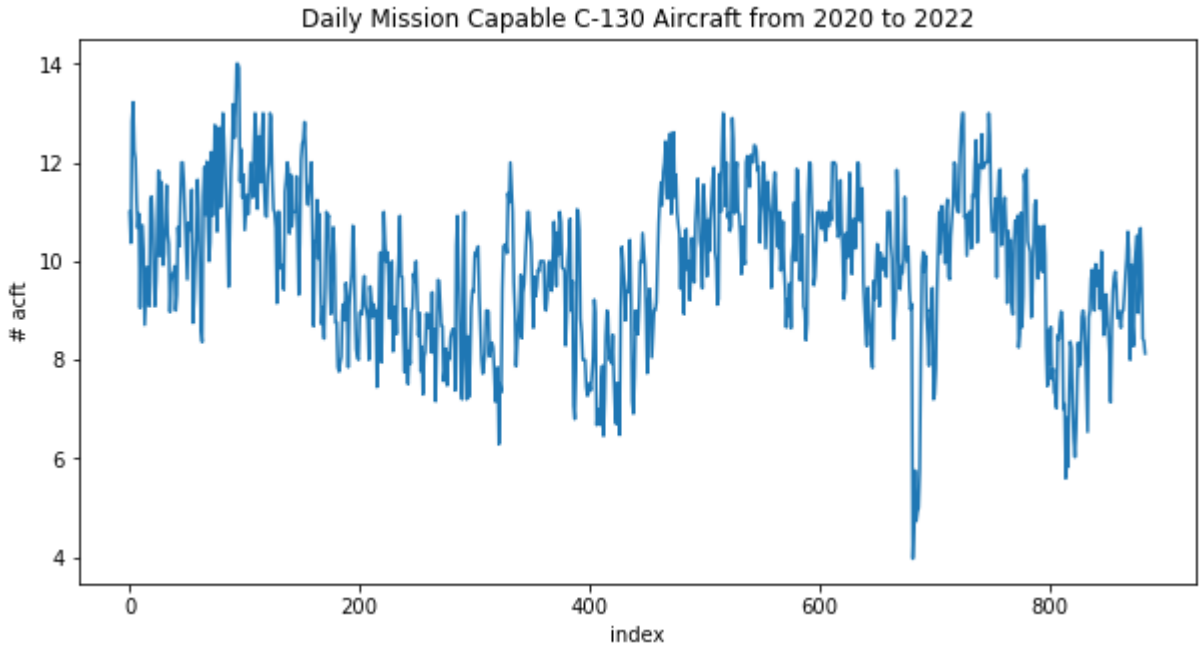
Further visualizing the daily MC aircraft using a histogram allows as to view the entire distribution and how the data is spread around the mean (10). As mentioned above, the distribution is slightly skewed to the left due to the outliers between 4 and 6.

In [5]:
```
# Show the outliers

outliers = df[df['mc_acft']< 6]
print(outliers)
```

	date	daily_poss	daily_mc	mc_acft	mc_rate	year	month	DoW
681	2021-11-12	312.0	95.2	3.97	0.305128	2021	Nov	Friday
682	2021-11-13	312.0	113.2	4.72	0.362821	2021	Nov	Saturday
683	2021-11-14	312.0	138.2	5.76	0.442949	2021	Nov	Sunday
684	2021-11-15	312.0	113.4	4.73	0.363462	2021	Nov	Monday
685	2021-11-16	312.0	118.0	4.92	0.378205	2021	Nov	Tuesday
686	2021-11-17	312.0	120.0	5.00	0.384615	2021	Nov	Wednesday
687	2021-11-18	312.0	138.4	5.77	0.443590	2021	Nov	Thursday
814	2022-03-25	288.0	134.3	5.60	0.466319	2022	Mar	Friday
816	2022-03-27	288.0	139.9	5.83	0.485764	2022	Mar	Sunday

Upon further investigation the outlier group was caused by number of C-130 aircraft grounded for two safety time changes -- aux pump wire TCTO1120. This explains the lower MC aircraft values in November. Let's further understand the impact of the outliers in our data by plotting the time-series graph.

In [23]:
```
# Plot the time-series data

def plot_df(df, x, y, title = "", xlabel='index', ylabel='# acft'):
    plt.figure(figsize=(10,5))
    plt.plot(x, y, color = 'tab:blue')
    plt.gca().set(title=title, xlabel=xlabel, ylabel=ylabel)
    plt.show()

plot_df(df, x=df.index, y=df.mc_acft,
        title = "Daily Mission Capable C-130 Aircraft from 2020 to 2022")
```



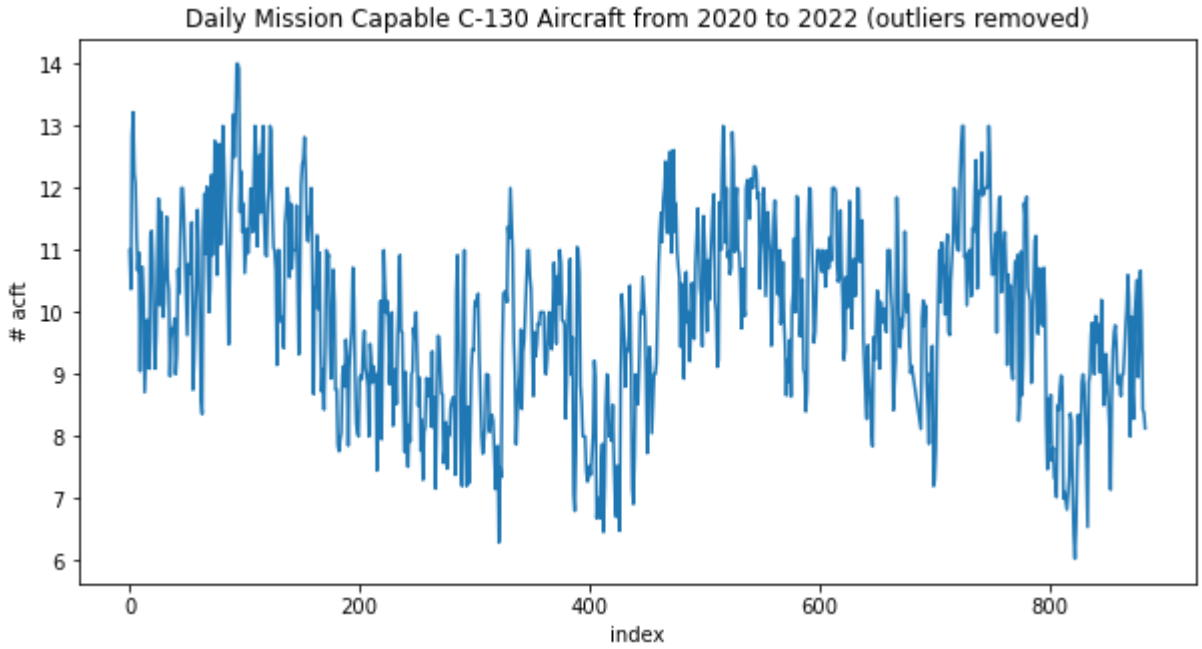
Outlier Treatment

So far, we have determined that outliers are present in our dataset. We can handle this in three different ways:

- drop the outlier
- create an outlier feature
- or dampen the impact of outlier by means of log transformation

For this analysis, let's see what the impact of dropping the outliers in the the dataset and the time-series.

In [22]:
```
# Let's reload the dataset in another dataframe

df2 = pd.read_csv('status_2020.csv')

df2 = df2[df2['mc_acft'] > 6 ]

# Plot the new time-series data without the outliers

def plot_df(df, x, y, title = "", xlabel='index', ylabel='# acft'):
    plt.figure(figsize=(10,5))
    plt.plot(x, y, color = 'tab:blue')
    plt.gca().set(title=title, xlabel=xlabel, ylabel=ylabel)
    plt.show()

plot_df(df2, x=df2.index, y=df2.mc_acft,
        title = "Daily Mission Capable C-130 Aircraft from 2020 to 2022 (outliers remov
```



After removing the outliers, our time-series is much cleaner. However, removing the outliers may not be appropriate in some occassions because we might be removing values that hold critical information about the data. We can instead mark the outliers by creating a new feature or by means of log transformation.

In [14]:
```
# Create a new feature

df['outlier'] = np.where(df['mc_acft'] > 6, 0, 1)

df.sample(10)
```

Out[14]:

	date	daily_poss	daily_mc	mc_acft	mc_rate	year	month	DoW	outlier
603	2021-08-26	312.0	264.0	11.00	0.846154	2021	Aug	Thursday	0
372	2021-01-07	336.0	254.0	10.58	0.755952	2021	Jan	Thursday	0
35	2020-02-05	312.0	215.2	8.97	0.689744	2020	Feb	Wednesday	0
197	2020-07-16	336.0	197.0	8.21	0.586310	2020	Jul	Thursday	0
61	2020-03-02	336.0	232.7	9.70	0.692560	2020	Mar	Monday	0
519	2021-06-03	336.0	288.0	12.00	0.857143	2021	Jun	Thursday	0
774	2022-02-13	312.0	205.0	8.54	0.657051	2022	Feb	Sunday	0
683	2021-11-14	312.0	138.2	5.76	0.442949	2021	Nov	Sunday	1
145	2020-05-25	336.0	281.2	11.72	0.836905	2020	May	Monday	0
641	2021-10-03	312.0	198.8	8.28	0.637179	2021	Oct	Sunday	0

In [18]:
```
# Finally, we can use log transformation on our dataset to dampen the impact of outlier

df["log_of_mc_acft"] = [np.log(x) for x in df["mc_acft"]]
```

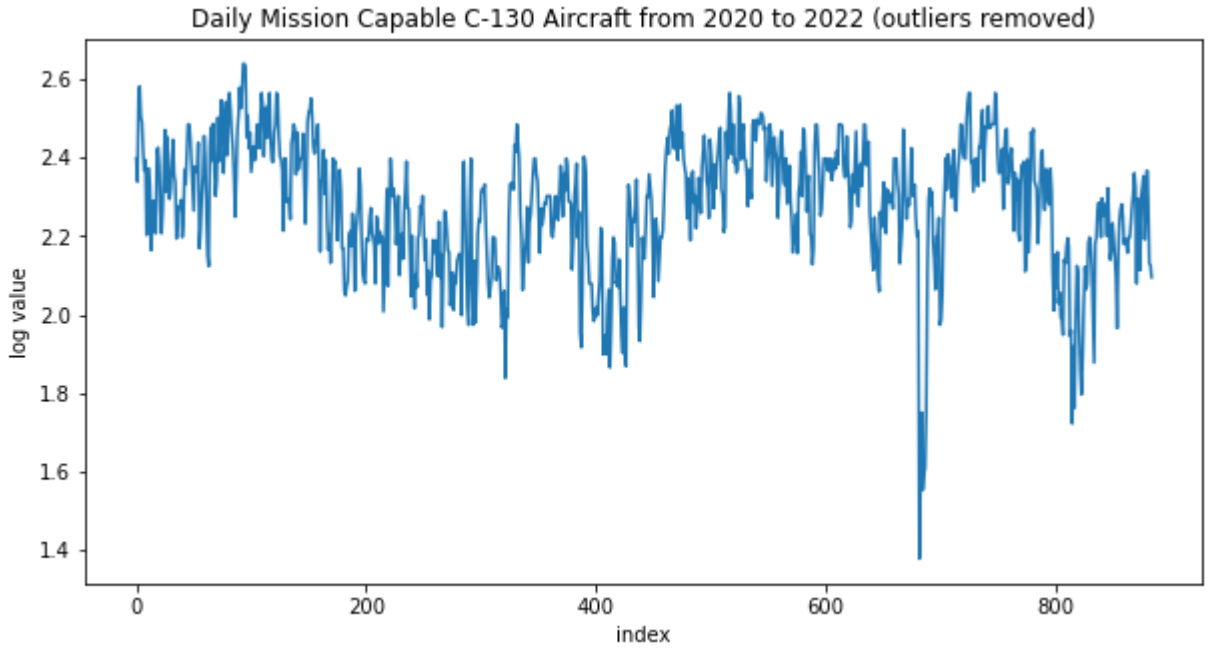
```
df.sample(10)
```

Out[18]:

	date	daily_poss	daily_mc	mc_acft	mc_rate	year	month	DoW	outlier	log_of_mc_acft
701	2021-12-02	312.0	187.4	7.81	0.600641	2021	Dec	Thursday	0	2.055405
150	2020-05-30	336.0	297.0	12.38	0.883929	2020	May	Saturday	0	2.516082
214	2020-08-02	336.0	216.0	9.00	0.642857	2020	Aug	Sunday	0	2.197225
401	2021-02-05	312.0	177.2	7.38	0.567949	2021	Feb	Friday	0	1.998774
276	2020-10-03	312.0	179.4	7.48	0.575000	2020	Oct	Saturday	0	2.012233
481	2021-04-26	336.0	244.8	10.20	0.728571	2021	Apr	Monday	0	2.322388
248	2020-09-05	312.0	229.8	9.58	0.736538	2020	Sep	Saturday	0	2.259678
681	2021-11-12	312.0	95.2	3.97	0.305128	2021	Nov	Friday	1	1.378766
584	2021-08-07	336.0	244.0	10.17	0.726190	2021	Aug	Saturday	0	2.319442
142	2020-05-22	336.0	264.0	11.00	0.785714	2020	May	Friday	0	2.397895

In [21]:

```
# Let's plot the log transformed MC aircraft

def plot_df(df, x, y, title = "", xlabel='index', ylabel='log value'):
    plt.figure(figsize=(10,5))
    plt.plot(x, y, color = 'tab:blue')
    plt.gca().set(title=title, xlabel=xlabel, ylabel=ylabel)
    plt.show()

plot_df(df, x=df.index, y=df.log_of_mc_acft,
        title = "Daily Mission Capable C-130 Aircraft from 2020 to 2022 (outliers remov
```



Summary

So far, we have explored the data of C-130 daily mission capable aircraft. The exploratory analysis revealed some key information such as the mean and median being alsmost equal, the slightly bigger spread of the distribution and the left-side skewness of the daily mc aircraft. We also found outlier days in Novemeber which was driven by TCTO inspections. Finally, we provided different ways to address these outliers to prepare for the next segment of this analysis, which is to develop the ARIMA forecasting model.